



Red Hat build of Apache Qpid ProtonJ2 1.1.0

Using Qpid ProtonJ2

Developing an AMQ messaging client using imperative Java

Red Hat build of Apache Qpid ProtonJ2 1.1.0 Using Qpid ProtonJ2

Developing an AMQ messaging client using imperative Java

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This guide describes how to install and use your client with other AMQ components.

Table of Contents

MAKING OPEN SOURCE MORE INCLUSIVE	4
CHAPTER 1. OVERVIEW	5
1.1. KEY FEATURES	5
1.2. SUPPORTED STANDARDS AND PROTOCOLS	5
1.3. SUPPORTED CONFIGURATIONS	5
1.4. TERMS AND CONCEPTS	5
1.5. DOCUMENT CONVENTIONS	6
The sudo command	6
File paths	6
Variable text	6
CHAPTER 2. INSTALLATION	7
2.1. PREREQUISITES	7
2.2. INSTALLING ON RED HAT ENTERPRISE LINUX	7
2.3. USING THE RED HAT MAVEN REPOSITORY	7
2.4. INSTALLING A LOCAL MAVEN REPOSITORY	7
2.5. INSTALLING THE EXAMPLES	8
CHAPTER 3. GETTING STARTED	9
3.1. PREREQUISITES	9
3.2. RUNNING HELLOWORLD ON RED HAT ENTERPRISE LINUX	9
3.3. RUNNING HELLO WORLD ON MICROSOFT WINDOWS	9
CHAPTER 4. EXAMPLES	10
4.1. SENDING MESSAGES	10
Running the example	11
4.2. RECEIVING MESSAGES	11
Running the example	12
CHAPTER 5. NETWORK CONNECTIONS	13
5.1. CONNECTION OPTIONS	13
5.1.1. Connection Transport Options	13
5.2. RECONNECT AND FAILOVER	13
CHAPTER 6. SECURITY	14
6.1. CONNECTING WITH A USER AND PASSWORD	14
6.2. CONFIGURING SASL AUTHENTICATION	14
6.3. CONFIGURING AN SSL/TLS TRANSPORT	14
CHAPTER 7. SENDERS AND RECEIVERS	16
7.1. CREATING QUEUES AND TOPICS ON DEMAND	16
7.2. CREATING DURABLE SUBSCRIPTIONS	16
7.3. CREATING SHARED SUBSCRIPTIONS	17
CHAPTER 8. MESSAGE DELIVERY	18
8.1. SENDING MESSAGES	18
8.2. TRACKING SENT MESSAGES	18
8.3. RECEIVING MESSAGES	18
8.4. ACKNOWLEDGING RECEIVED MESSAGES	18
CHAPTER 9. LOGGING	20
9.1. CONFIGURING A SIMPLE LOGGER WITH SLF4J	20
9.2. ENABLING PROTOCOL LOGGING	20

CHAPTER 10. INTEROPERABILITY	21
10.1. INTEROPERATING WITH OTHER AMQP CLIENTS	21
10.2. INTEROPERATING WITH RED HAT BUILD OF APACHE QPID JMS	25
JMS message types	25
10.3. CONNECTING TO AMQ BROKER	26
APPENDIX A. USING YOUR SUBSCRIPTION	27
A.1. ACCESSING YOUR ACCOUNT	27
A.2. ACTIVATING A SUBSCRIPTION	27
A.3. DOWNLOADING RELEASE FILES	27
A.4. REGISTERING YOUR SYSTEM FOR PACKAGES	27
APPENDIX B. USING RED HAT MAVEN REPOSITORIES	29
B.1. USING THE ONLINE REPOSITORY	29
Adding the repository to your Maven settings	29
Adding the repository to your POM file	30
B.2. USING A LOCAL REPOSITORY	30
APPENDIX C. USING AMQ BROKER WITH THE EXAMPLES	32
C.1. INSTALLING THE BROKER	32
C.2. STARTING THE BROKER	32
C.3. CREATING A QUEUE	32
C.4. STOPPING THE BROKER	33

MAKING OPEN SOURCE MORE INCLUSIVE

Red Hat is committed to replacing problematic language in our code, documentation, and web properties. We are beginning with these four terms: master, slave, blacklist, and whitelist. Because of the enormity of this endeavor, these changes will be implemented gradually over several upcoming releases. For more details, see [our CTO Chris Wright's message](#).

CHAPTER 1. OVERVIEW

Red Hat build of Apache Qpid ProtonJ2 is a lightweight AMQP 1.0 library for the Java platform. It enables you to write Java applications that send and receive AMQP messages.

Red Hat build of Apache Qpid ProtonJ2 is part of AMQ Clients, a suite of messaging libraries supporting multiple languages and platforms. For information on available clients, see [AMQ Clients](#).

1.1. KEY FEATURES

- SSL/TLS for secure communication
- Flexible SASL authentication
- Seamless conversion between AMQP and native data types
- Access to all the features and capabilities of AMQP 1.0
- An integrated development environment with full *IntelliSense* API documentation

1.2. SUPPORTED STANDARDS AND PROTOCOLS

Red Hat build of Apache Qpid ProtonJ2 supports the following industry-recognized standards and network protocols:

- Version 1.0 of the [Advanced Message Queuing Protocol](#) (AMQP)
- Versions 1.0, 1.1, 1.2, and 1.3 of the [Transport Layer Security](#) (TLS) protocol, the successor to SSL
- [Simple Authentication and Security Layer](#) (SASL) mechanisms ANONYMOUS, PLAIN, and EXTERNAL
- Modern [TCP](#) with [IPv6](#)

1.3. SUPPORTED CONFIGURATIONS

Refer to [Red Hat AMQ Supported Configurations](#) on the Red Hat Customer Portal for current information regarding Red Hat build of Apache Qpid ProtonJ2 supported configurations.

1.4. TERMS AND CONCEPTS

This section introduces the core API entities and describes how they operate together.

Table 1.1. API terms

Entity	Description
Client	A context for creating Connections to Brokers
Connection	A channel for communication between two peers on a network
Session	A context for sending and receiving messages

Entity	Description
Sender	A channel for sending messages to a target
Receiver	A channel for receiving messages from a source
Delivery	A Delivery received from a IReceiver that can be acted upon and contains a message
Message	A mutable holder of application data

Red Hat build of Apache Qpid ProtonJ2 sends and receives *messages*. Messages are transferred to Brokers via *Senders* and from Brokers using *Receivers*. Received Messages are wrapped in a Delivery which can be Accepted, Rejected or Released.

Senders and *Receivers* are created using *Connections* which are in turn created using *Clients*. *Sessions* are established over *Connections*.

1.5. DOCUMENT CONVENTIONS

The **sudo** command

In this document, **sudo** is used for any command that requires root privileges. Exercise caution when using **sudo** because any changes can affect the entire system. For more information about **sudo**, see [Using the sudo command](#).

File paths

In this document, all file paths are valid for Linux, UNIX, and similar operating systems (for example, **/home/andrea**). On Microsoft Windows, you must use the equivalent Windows paths (for example, **C:\Users\andrea**).

Variable text

This document contains code blocks with variables that you must replace with values specific to your environment. Variable text is enclosed in arrow braces and styled as italic monospace. For example, in the following command, replace **<project-dir>** with the value for your environment:

```
$ cd <project-dir>
```

CHAPTER 2. INSTALLATION

This chapter guides you through the steps to install Red Hat build of Apache Qpid ProtonJ2 in your environment.

2.1. PREREQUISITES

- You must have a [subscription](#) to access AMQ release files and repositories.
- To build programs with Red Hat build of Apache Qpid ProtonJ2, you must install [Apache Maven](#).
- To use Red Hat build of Apache Qpid ProtonJ2, you must install Java.

2.2. INSTALLING ON RED HAT ENTERPRISE LINUX

2.3. USING THE RED HAT MAVEN REPOSITORY

Procedure

Configure your Maven environment to download the client library from the Red Hat Maven repository.

Procedure

1. Add the Red Hat repository to your Maven settings or POM file. For example configuration files, see [Section B.1, "Using the online repository"](#).

```
<repository>
  <id>red-hat-ga</id>
  <url>https://maven.repository.redhat.com/ga</url>
</repository>
```

2. Add the library dependency to your POM file.

```
<dependency>
  <groupId>org.apache.qpid</groupId>
  <artifactId>protonj2-client</artifactId>
  <version>1.1.0.redhat-00001</version>
</dependency>
```

The client is now available in your Maven project.

2.4. INSTALLING A LOCAL MAVEN REPOSITORY

As an alternative to the online repository, Red Hat build of Apache Qpid ProtonJ2 can be installed to your local filesystem as a file-based Maven repository.

Procedure

1. [Use your subscription](#) to download the **amq-qpid-protonj2-1.1.0 Maven repository.zip** file.
2. Extract the file contents into a directory of your choosing.
On Linux or UNIX, use the **unzip** command to extract the file contents.

```
$ unzip amq-qpid-protonj2-1.1.0-maven-repository.zip
```

On Windows, right-click the .zip file and select **Extract All**.

3. Configure Maven to use the repository in the **maven-repository** directory inside the extracted install directory. For more information, see [Section B.2, "Using a local repository"](#).

2.5. INSTALLING THE EXAMPLES

1. Use the **git clone** command to clone the source repository to a local directory named **qpid-protonj2**:

```
$ git clone https://github.com/apache/qpid-protonj2.git
```

Change to the **qpid-protonj2** directory and use the git checkout command to checkout the commit associated with this release:

```
$ cd qpid-protonj2
$ git checkout 1.1,0
$ cd protonj2-client-examples
```

The resulting local directory is referred to as **<source-dir>** in this guide.

CHAPTER 3. GETTING STARTED

This chapter guides you through the steps to set up your environment and run a simple messaging program.

3.1. PREREQUISITES

- You must complete the [installation](#) procedure for your environment.
- You must have an AMQP 1.0 message broker listening for connections on interface **localhost** and port **5672**. It must have anonymous access enabled. For more information, see [Starting the broker](#).
- You must have a queue named **hello-world-example**. For more information, see [Creating a queue](#).
- Alternatively, you can specify above information using following environment variables: **HOST**, **PORT**, **ADDRESS**

3.2. RUNNING HELLOWORLD ON RED HAT ENTERPRISE LINUX

The Hello World example creates a connection to the broker, sends a message containing a greeting to the **hello-world-example** queue, and receives it back. On success, it prints the received message to the console.

1. Use maven to build the module, and additionally copy the dependencies alongside their output:

```
cd <source-dir>
mvn clean package dependency:copy-dependencies -DincludeScope=runtime -DskipTests
```

2. Run the examples:

```
$ java -cp "target/classes/:target/dependency/*"
org.apache.qpid.protonj2.client.examples.HelloWorld
```

3.3. RUNNING HELLO WORLD ON MICROSOFT WINDOWS

The Hello World example creates a connection to the broker, sends a message containing a greeting to the **hello-world-example** queue, and receives it back. On success, it prints the received message to the console.

1. Use maven to build the module, and additionally copy the dependencies alongside their output:

```
cd <source-dir>
mvn clean package dependency:copy-dependencies -DincludeScope=runtime -DskipTests
```

2. Run the examples:

```
$ java -cp "target\classes\;target\dependency\*"
org.apache.qpid.protonj2.client.examples.HelloWorld
```

CHAPTER 4. EXAMPLES

This chapter demonstrates the use of Red Hat build of Apache Qpid ProtonJ2 through example programs.

For more examples, see the [Apache Qpid ProtonJ2 examples](#).

4.1. SENDING MESSAGES

This client program connects to a server using **<serverHost>** and **<serverPort>**, creates a sender for target **<address>**, sends 100 messages containing **String** and exits.

Example: Sending messages

```
package org.apache.qpid.protonj2.client.examples;

import org.apache.qpid.protonj2.client.Client;
import org.apache.qpid.protonj2.client.Connection;
import org.apache.qpid.protonj2.client.ConnectionOptions;
import org.apache.qpid.protonj2.client.Message;
import org.apache.qpid.protonj2.client.Sender;
import org.apache.qpid.protonj2.client.Tracker; ❶

public class Send {

    private static final int MESSAGE_COUNT = 100; ❷

    public static void main(String[] argv) throws Exception {
        final String serverHost = System.getProperty("HOST", "localhost"); ❸
        final int serverPort = Integer.getInteger("PORT", 5672); ❹
        final String address = System.getProperty("ADDRESS", "send-receive-example");

        final Client client = Client.create(); ❺

        final ConnectionOptions options = new ConnectionOptions(); ❻
        options.user(System.getProperty("USER"));
        options.password(System.getProperty("PASSWORD"));

        try (Connection connection = client.connect(serverHost, serverPort, options); ❼
            Sender sender = connection.openSender(address)) { ❽

            for (int i = 0; i < MESSAGE_COUNT; ++i) {
                Message<String> message = Message.create(String.format("Hello World! [%s]", i)); ❾
                Tracker tracker = sender.send(message); ❿
                tracker.awaitSettlement(); ⓫

                System.out.println(String.format("Sent message to %s: %s", sender.address(),
message.body()));
            }
        }
    }
}
```

<.> packages required by Red Hat build of Apache Qpid ProtonJ2.

<.> The number of messages to send.

<.> **serverHost** is the network address of the host or virtual host for the AMQP connection and can be configured by setting the Environment variable **HOST**.

<.> **serverPort** is the port on the host that the broker is accepting connections and can be configured by setting the environment variable **PORT**.

<.> **Client** is the container that can create multiple Connections to a broker.

<.> **options** is used for various setting, including **User** and **Password**. See [Section 5.1, "Connection Options"](#) for more information.

<.> **connection** is the AMQP Connection to a broker.

<.> Create a **sender** for transferring messages to the broker.

<.> In the message send loop a new message is created.

<.> The message is sent to the broker.

<.> Wait for the broker to settle the message.

Running the example

To run the example program, see [Chapter 3, Getting started](#).

4.2. RECEIVING MESSAGES

This client program connects to a server using **<connection-url>**, creates a receiver for source **<address>**, and receives messages until it is terminated or it reaches **<count>** messages.

Example: Receiving messages

```
package org.apache.qpid.protonj2.client.examples;

import org.apache.qpid.protonj2.client.Client;
import org.apache.qpid.protonj2.client.Connection;
import org.apache.qpid.protonj2.client.ConnectionOptions;
import org.apache.qpid.protonj2.client.Delivery;
import org.apache.qpid.protonj2.client.Message;
import org.apache.qpid.protonj2.client.Receiver; 1

public class Receive {

    private static final int MESSAGE_COUNT = 100; 2

    public static void main(String[] args) throws Exception {
        final String serverHost = System.getProperty("HOST", "localhost"); 3
        final int serverPort = Integer.getInteger("PORT", 5672); 4
        final String address = System.getProperty("ADDRESS", "send-receive-example"); 5

        final Client client = Client.create(); 6
    }
}
```

```

final ConnectionOptions options = new ConnectionOptions(); 7
options.user(System.getProperty("USER"));
options.password(System.getProperty("PASSWORD"));

try (Connection connection = client.connect(serverHost, serverPort, options); 8
    Receiver receiver = connection.openReceiver(address)) { 9

    for (int i = 0; i < MESSAGE_COUNT; ++i) {
        Delivery delivery = receiver.receive(); 10
        Message<String> message = delivery.message(); 11

        System.out.println("Received message with body: " + message.body());
    }
}
}
}

```

- 1 1 required packages for Red Hat build of Apache Qpid ProtonJ2.
- 2 2 The number of messages to receive.
- 3 3 **serverHost** is the network address of the host or virtual host for the AMQP connection and can be configured by setting the Environment variable **HOST**.
- 4 4 **serverPort** is the port on the host that the broker is accepting connections and can be configured by setting the environment variable **PORT**.
- 5 5 6 **Client** is the container that can create multiple Connections to a broker.
- 6 7 **options** is used for various setting, including **User** and **Password**. See [Section 5.1, "Connection Options"](#) for more information.
- 7 8 **connection** is the AMQP Connection to a broker.
- 8 9 Create a **receiver** for receiving messages from the broker.
- 9 10 In the message receive loop a new delivery is received.
- 10 11 The message is obtained from the **delivery**.

Running the example

To run the example program, see [Chapter 3, Getting started](#).

CHAPTER 5. NETWORK CONNECTIONS

5.1. CONNECTION OPTIONS

This section describes how to configure how to configure connections.

The *ConnectionOptions* object can be provided to a Client instance when creating a new connection and allows configuration of several different aspects of the resulting Connection instance.

ConnectionOptions can be passed in the connect method on *IClient* and are used to configure

Example: Configuring authentication

```
final ConnectionOptions connectionOptions = new ConnectionOptions();
connectionOptions.user(System.getProperty("USER"));
connectionOptions.password(System.getProperty("PASSWORD"));
Connection connection = client.connect(serverHost, serverPort, connectionOptions);
```

For a definitive list of options refer to [ConnectionOptions](#)

5.1.1. Connection Transport Options

The ConnectionOptions object exposes a set of configuration options for the underlying I/O transport layer known as the TransportOptions which allows for fine grained configuration of network level options.

Example: Configuring transport options

```
final ConnectionOptions connectionOptions = new ConnectionOptions();
connectionOptions.transportOptions().tcpNoDelay(false);
Connection connection = client.connect(serverHost, serverPort, connectionOptions);
```

For a definitive list of options refer to [TransportOptions](#)

5.2. RECONNECT AND FAILOVER

When creating a new connection it is possible to configure that connection to perform automatic connection recovery.

Example: Configuring transport reconnection and failover

```
final ConnectionOptions connectionOptions = new ConnectionOptions();
connectionOptions.reconnectOptions().reconnectEnabled(true);
connectionOptions.reconnectOptions().reconnectDelay(30000);
connectionOptions.reconnectOptions().addReconnectLocation(hostname, port);
Connection connection = client.connect(serverHost, serverPort, connectionOptions);
```

For a definitive list of options refer to [ReconnectionOptions](#)

CHAPTER 6. SECURITY

6.1. CONNECTING WITH A USER AND PASSWORD

Red Hat build of Apache Qpid ProtonJ2 can authenticate connections with a user and password.

To specify the credentials used for authentication, set the **Username** and **Password** fields on the *ConnectionOptions* object

Example: Connecting with a user and password

```
ConnectionOptions connectionOptions = new ConnectionOptions();
connectionOptions.user("user");
connectionOptions.password("pass");
```

6.2. CONFIGURING SASL AUTHENTICATION

Client connections to remote peers may exchange SASL user name and password credentials. The presence of the *user* field in the connection URI controls this exchange. If *user* is specified then SASL credentials are exchanged; if *user* is absent then the SASL credentials are not exchanged.

Various SASL mechanisms are supported, please see [SASL Reference](#)

Example: Adding an allowed mechanism

```
ConnectionOptions connectionOptions = new ConnectionOptions();
connectionOptions.user(System.getProperty("USER"));
connectionOptions.password(System.getProperty("PASSWORD"));
connectionOptions.saslOptions().saslEnabled(true);
connectionOptions.saslOptions().addAllowedMechanism("PLAIN")
```

6.3. CONFIGURING AN SSL/TLS TRANSPORT

SSL/TLS can be configured programmatically as follows.

Example: Configuring SSL

```
ConnectionOptions connectionOptions = new ConnectionOptions();
connectionOptions.sslOptions().sslEnabled(true);
connectionOptions.sslOptions().keyAlias("alias");
connectionOptions.sslOptions().keyStoreLocation("keystore-location");
connectionOptions.sslOptions().keyStorePassword("password");
connectionOptions.sslOptions().trustStoreLocation("trustore-location");
```

For a full list of options refer to [SSL Options](#)

Alternatively the trust store and keystore configuration can be set using the following environmental properties.

- **javax.net.ssl.keyStore** The location of the keystore
- **javax.net.ssl.keyStoreType** The type of the keystore

- **javax.net.ssl.keyStorePassword** The keystore password
- **javax.net.ssl.trustStore** The location of the truststore
- **javax.net.ssl.trustStoreType** The truststore type
- **javax.net.ssl.trustStorePassword** The truststore password



NOTE

[TLS Server Name Indication](#) (SNI) is handled automatically by the client library. However, SNI is signaled only for addresses that use the *amqps* transport scheme where the host is a fully qualified domain name or a host name. SNI is not signaled when the host is a numeric IP address.

CHAPTER 7. SENDERS AND RECEIVERS

The client uses sender and receiver links to represent channels for delivering messages. Senders and receivers are unidirectional, with a source end for the message origin, and a target end for the message destination.

Sources and targets often point to queues or topics on a message broker. Sources are also used to represent subscriptions.

7.1. CREATING QUEUES AND TOPICS ON DEMAND

Some message servers support on-demand creation of queues and topics. When a sender or receiver is attached, the server uses the sender target address or the receiver source address to create a queue or topic with a name matching the address.

The message server typically defaults to creating either a queue (for one-to-one message delivery) or a topic (for one-to-many message delivery). The client can indicate which it prefers by setting the **queue** or **topic** capability on the source or target.

To select queue or topic semantics, follow these steps:

1. Configure your message server for automatic creation of queues and topics. This is often the default configuration.
2. Set either the **queue** or **topic** capability on your sender target or receiver source, as in the examples below.

Example: Sending to a queue created on demand

```
SenderOptions senderOptions = new SenderOptions();
senderOptions.targetOptions().capabilities("queue");
Sender sender = connection.openSender(address, senderOptions);
```

Example: Receiving from a topic created on demand

```
ReceiverOptions receiverOptions = new ReceiverOptions();
receiverOptions.sourceOptions().capabilities("topic");
Receiver receiver = connection.openReceiver(address, receiverOptions);
```

7.2. CREATING DURABLE SUBSCRIPTIONS

A durable subscription is a piece of state on the remote server representing a message receiver. Ordinarily, message receivers are discarded when a client closes. However, because durable subscriptions are persistent, clients can detach from them and then re-attach later. Any messages received while detached are available when the client re-attaches.

Durable subscriptions are uniquely identified by combining the client container ID and receiver name to form a subscription ID. These must have stable values so that the subscription can be recovered.

To create a durable subscription, follow these steps:

1. Set the connection container ID to a stable value, such as **client-1**:

```
ClientOptions clientOptions = new ClientOptions();
```

```
clientOptions.id("client-1");
Client client = Client.Create(clientOptions);
```

2. Configure the receiver to act like a topic subscription.

```
ReceiverOptions receiverOptions = new ReceiverOptions();
receiverOptions.sourceOptions().capabilities("topic");
receiverOptions.sourceOptions().durabilityMode(DurabilityMode.UNSETTLED_STATE);
receiverOptions.sourceOptions().expiryPolicy(ExpiryPolicy.NEVER);
Receiver receiver = connection.openDurableReceiver(address, "sub-1", receiverOptions);
```

To detach from a subscription, close the receiver.

1. Delete the subscription.

```
receiver.closeAsync();
```

if you dont close the receiver then the durable subscription will remain.

7.3. CREATING SHARED SUBSCRIPTIONS

A shared subscription is a piece of state on the remote server representing one or more message receivers. Because it is shared, multiple clients can consume from the same stream of messages.

The client configures a shared subscription by setting the **shared** capability on the receiver source.

Shared subscriptions are uniquely identified by combining the client container ID and receiver name to form a subscription ID. These must have stable values so that multiple client processes can locate the same subscription. If the **global** capability is set in addition to **shared**, the receiver name alone is used to identify the subscription.

To create a shared subscription, follow these steps:

1. Set the connection container ID to a stable value, such as **client-1**:

```
ClientOptions clientOptions = new ClientOptions();
clientOptions.id("client-1");
Client client = Client.Create(clientOptions);
```

2. Configure the receiver for sharing by setting the **shared** capability:

```
ReceiverOptions receiverOptions = new ReceiverOptions();
receiverOptions.sourceOptions().capabilities("topic", "shared");
receiverOptions.sourceOptions().durabilityMode(DurabilityMode.UNSETTLED_STATE);
receiverOptions.sourceOptions().expiryPolicy(ExpiryPolicy.NEVER);
Receiver receiver = connection.openDurableReceiver(address, "sub-1", receiverOptions);
```

CHAPTER 8. MESSAGE DELIVERY

8.1. SENDING MESSAGES

To send a message, create a client, connection and a sender

Example: Sending messages

```
Connection connection = client.connect(serverHost, serverPort, options);
SenderOptions senderOptions = new SenderOptions();
Sender sender = connection.openSender(address, senderOptions);
Message<String> message = Message.create("Hello World!");
Tracker tracker = sender.send(message);
```

For more information, see the [Send.java example](#).

8.2. TRACKING SENT MESSAGES

When the message is sent a Tracker is returned which can be used to track the message or settle it if not using auto settlement

Example: waiting for the broker to settle the message

```
tracker.awaitSettlement();
```

Example: settling the message

```
tracker.settle();
```

8.3. RECEIVING MESSAGES

To receive a message, create a connection, client and receiver.

Example: Receiving messages

```
Connection connection = client.connect(serverHost, serverPort, options);
Receiver receiver = connection.openReceiver(address);
Delivery delivery = receiver.receive();
Message<object> received = delivery.message();
```

The **Receiver.accept()** call tells the remote peer that the message was received and processed.

For more information, see the [Receive.java example](#).

8.4. ACKNOWLEDGING RECEIVED MESSAGES

The *Delivery* object can be used to accept, reject modify the delivery.

Example: Acknowledging received messages

```
----
```

```
delivery.accept()
```

```
----
```

Example: Rejecting received messages

```
----
```

```
delivery.reject()
```

```
----
```

Example: Releasing received messages

```
----
```

```
delivery.Release()
```

```
----
```

CHAPTER 9. LOGGING

Logging is important in troubleshooting and debugging. protonJ2 uses slf4j as its logger for full configuration options see [Slf4j Manual](#).

9.1. CONFIGURING A SIMPLE LOGGER WITH SLF4J

To configure a simple logger using slf4j create a file ``simplelogger.properties` and configure like so:

Example Simple Logger

```
org.slf4j.simpleLogger.logFile=System.out

org.slf4j.simpleLogger.defaultLogLevel=warn

org.slf4j.simpleLogger.showDateTime=true
org.slf4j.simpleLogger.dateTimeFormat=yyyy-MM-dd HH:mm:ss,SSS

org.slf4j.simpleLogger.log.org.apache.qpid.protonj2=warn
org.slf4j.simpleLogger.log.org.apache.qpid.protonj2.client=warn
```

9.2. ENABLING PROTOCOL LOGGING

The following code writes AMQP frames to the console.

Example: Logging AMQP frames

```
ConnectionOptions options = new ConnectionOptions();
options.traceFrames(true);
```

Alternatively you get set the environment variable **PN_TRACE_FRM** to true;

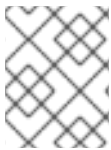
CHAPTER 10. INTEROPERABILITY

This chapter discusses how to use Red Hat build of Apache Qpid ProtonJ2 in combination with other AMQ components. For an overview of the compatibility of AMQ components, see the [product introduction](#).

10.1. INTEROPERATING WITH OTHER AMQP CLIENTS

AMQP messages are composed using the [AMQP type system](#). This common format is one of the reasons AMQP clients in different languages are able to interoperate with each other.

When sending messages, Red Hat build of Apache Qpid ProtonJ2 automatically converts language-native types to AMQP-encoded data. When receiving messages, the reverse conversion takes place.



NOTE

More information about AMQP types is available at the [interactive type reference](#) maintained by the Apache Qpid project.

Table 10.1. AMQP types

AMQP type	Description
null	An empty value
boolean	A true or false value
char	A single Unicode character
string	A sequence of Unicode characters
binary	A sequence of bytes
byte	A signed 8-bit integer
short	A signed 16-bit integer
int	A signed 32-bit integer
long	A signed 64-bit integer
ubyte	An unsigned 8-bit integer
ushort	An unsigned 16-bit integer
uint	An unsigned 32-bit integer
ulong	An unsigned 64-bit integer

AMQP type	Description
float	A 32-bit floating point number
double	A 64-bit floating point number
array	A sequence of values of a single type
list	A sequence of values of variable type
map	A mapping from distinct keys to values
uuid	A universally unique identifier
symbol	A 7-bit ASCII string from a constrained domain
timestamp	An absolute point in time

Table 10.2. Qpid ProtonJ2 types before encoding and after decoding

AMQP type	Qpid ProtonJ2 type before encoding	Qpid ProtonJ2 type after decoding
null	null	null
boolean	System.Boolean	System.Boolean
char	System.Char	System.Char
string	System.String	System.String
binary	System.Byte[]	System.Byte[]
byte	System.SByte	System.SByte
short	System.Int16	System.Int16
int	System.Int32	System.Int32
long	System.Int64	System.Int64
ubyte	System.Byte	System.Byte
ushort	System.UInt16	System.UInt16
uint	System.UInt32	System.UInt32

AMQP type	Qpid ProtonJ2 type before encoding	Qpid ProtonJ2 type after decoding
ulong	System.UInt64	System.UInt64
float	System.Single	System.Single
double	System.Double	System.Double
list	Amqp.List	Amqp.List
map	Amqp.Map	Amqp.Map
uuid	System.Guid	System.Guid
symbol	Amqp.Symbol	Amqp.Symbol
timestamp	System.DateTime	System.DateTime

Table 10.3. Qpid ProtonJ2 and other AMQ client types (1 of 2)

Qpid ProtonJ2 type before encoding	AMQ C++ type	Red Hat build of Rhea type
null	nullptr	null
System.Boolean	bool	boolean
System.Char	wchar_t	number
System.String	std::string	string
System.Byte[]	proton::binary	string
System.SByte	int8_t	number
System.Int16	int16_t	number
System.Int32	int32_t	number
System.Int64	int64_t	number
System.Byte	uint8_t	number
System.UInt16	uint16_t	number
System.UInt32	uint32_t	number

Qpid ProtonJ2 type before encoding	AMQ C++ type	Red Hat build of Rhea type
System.UInt64	uint64_t	number
System.Single	float	number
System.Double	double	number
Amqp.List	std::vector	Array
Amqp.Map	std::map	object
System.Guid	proton::uuid	number
Amqp.Symbol	proton::symbol	string
System.DateTime	proton::timestamp	number

Table 10.4. Qpid ProtonJ2 and other AMQ client types (2 of 2)

Qpid ProtonJ2	Red Hat build of Apache Qpid Proton Python	
null	None	
System.Boolean	bool	
System.Char	proton.char	
System.String	unicode	
System.Byte[]	bytes	
System.SByte	proton.byte	
System.Int16	proton.short	
System.Int32	proton.int32	
System.Int64	long	
System.Byte	proton.ubyte	
System.UInt16	proton.ushort	

Qpid ProtonJ2	Red Hat build of Apache Qpid Proton Python	
System.UInt32	proton.uint	
System.UInt64	proton.ulong	
System.Single	proton.float32	
System.Double	float	
Amqp.List	list	
Amqp.Map	dict	
System.Guid	uuid.UUID	
Amqp.Symbol	proton.symbol	
System.DateTime	proton.timestamp	

10.2. INTEROPERATING WITH RED HAT BUILD OF APACHE QPID JMS

AMQP defines a standard mapping to the JMS messaging model. This section discusses the various aspects of that mapping. For more information, see the Red Hat build of Apache Qpid JMS [Interoperability](#) chapter.

JMS message types

Red Hat build of Apache Qpid ProtonJ2 provides a single message type whose body type can vary. By contrast, the JMS API uses different message types to represent different kinds of data. The table below indicates how particular body types map to JMS message types.

For more explicit control of the resulting JMS message type, you can set the **x-opt-jms-msg-type** message annotation.

Table 10.5. Red Hat build of Apache Qpid ProtonJ2 and JMS message types

Red Hat build of Apache Qpid ProtonJ2 body type	JMS message type
System.String	TextMessage
null	TextMessage
System.Byte[]	BytesMessage
Any other type	ObjectMessage

10.3. CONNECTING TO AMQ BROKER

AMQ Broker is designed to interoperate with AMQP 1.0 clients. Check the following to ensure the broker is configured for AMQP messaging:

- Port 5672 in the network firewall is open.
- The AMQ Broker AMQP acceptor is enabled. See [Default acceptor settings](#).
- The necessary addresses are configured on the broker. See [Addresses, and Queues](#).
- The broker is configured to permit access from your client, and the client is configured to send the required credentials. See [Broker Security](#).

APPENDIX A. USING YOUR SUBSCRIPTION

AMQ is provided through a software subscription. To manage your subscriptions, access your account at the Red Hat Customer Portal.

A.1. ACCESSING YOUR ACCOUNT

Procedure

1. Go to access.redhat.com.
2. If you do not already have an account, create one.
3. Log in to your account.

A.2. ACTIVATING A SUBSCRIPTION

Procedure

1. Go to access.redhat.com.
2. Navigate to **My Subscriptions**.
3. Navigate to **Activate a subscription** and enter your 16-digit activation number.

A.3. DOWNLOADING RELEASE FILES

To access .zip, .tar.gz, and other release files, use the customer portal to find the relevant files for download. If you are using RPM packages or the Red Hat Maven repository, this step is not required.

Procedure

1. Open a browser and log in to the Red Hat Customer Portal **Product Downloads** page at access.redhat.com/downloads.
2. Locate the **Red Hat AMQ** entries in the **INTEGRATION AND AUTOMATION** category.
3. Select the desired AMQ product. The **Software Downloads** page opens.
4. Click the **Download** link for your component.

A.4. REGISTERING YOUR SYSTEM FOR PACKAGES

To install RPM packages for this product on Red Hat Enterprise Linux, your system must be registered. If you are using downloaded release files, this step is not required.

Procedure

1. Go to access.redhat.com.
2. Navigate to **Registration Assistant**.
3. Select your OS version and continue to the next page.

4. Use the listed command in your system terminal to complete the registration.

For more information about registering your system, see one of the following resources:

- [Red Hat Enterprise Linux 8 - Registering the system and managing subscriptions](#)
- [Red Hat Enterprise Linux 9 - Registering the system and managing subscriptions](#)

APPENDIX B. USING RED HAT MAVEN REPOSITORIES

This section describes how to use Red Hat-provided Maven repositories in your software.

B.1. USING THE ONLINE REPOSITORY

Red Hat maintains a central Maven repository for use with your Maven-based projects. For more information, see the [repository welcome page](#).

There are two ways to configure Maven to use the Red Hat repository:

- [Add the repository to your Maven settings](#)
- [Add the repository to your POM file](#)

Adding the repository to your Maven settings

This method of configuration applies to all Maven projects owned by your user, as long as your POM file does not override the repository configuration and the included profile is enabled.

Procedure

1. Locate the Maven **settings.xml** file. It is usually inside the **.m2** directory in the user home directory. If the file does not exist, use a text editor to create it.
On Linux or UNIX:

```
/home/<username>/.m2/settings.xml
```

On Windows:

```
C:\Users\<username>\.m2\settings.xml
```

2. Add a new profile containing the Red Hat repository to the **profiles** element of the **settings.xml** file, as in the following example:

Example: A Maven settings.xml file containing the Red Hat repository

```
<settings>
  <profiles>
    <profile>
      <id>red-hat</id>
      <repositories>
        <repository>
          <id>red-hat-ga</id>
          <url>https://maven.repository.redhat.com/ga</url>
        </repository>
      </repositories>
      <pluginRepositories>
        <pluginRepository>
          <id>red-hat-ga</id>
          <url>https://maven.repository.redhat.com/ga</url>
          <releases>
            <enabled>true</enabled>
          </releases>
          <snapshots>
```

```

        <enabled>false</enabled>
      </snapshots>
    </pluginRepository>
  </pluginRepositories>
</profile>
</profiles>
<activeProfiles>
  <activeProfile>red-hat</activeProfile>
</activeProfiles>
</settings>

```

For more information about Maven configuration, see the [Maven settings reference](#).

Adding the repository to your POM file

To configure a repository directly in your project, add a new entry to the **repositories** element of your POM file, as in the following example:

Example: A Maven pom.xml file containing the Red Hat repository

```

<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>example-app</artifactId>
  <version>1.0.0</version>

  <repositories>
    <repository>
      <id>red-hat-ga</id>
      <url>https://maven.repository.redhat.com/ga</url>
    </repository>
  </repositories>
</project>

```

For more information about POM file configuration, see the [Maven POM reference](#).

B.2. USING A LOCAL REPOSITORY

Red Hat provides file-based Maven repositories for some of its components. These are delivered as downloadable archives that you can extract to your local filesystem.

To configure Maven to use a locally extracted repository, apply the following XML in your Maven settings or POM file:

```

<repository>
  <id>red-hat-local</id>
  <url>${repository-url}</url>
</repository>

```

`${repository-url}` must be a file URL containing the local filesystem path of the extracted repository.

Table B.1. Example URLs for local Maven repositories

Operating system	Filesystem path	URL
Linux or UNIX	/home/alice/maven-repository	file:/home/alice/maven-repository
Windows	C:\repos\red-hat	file:C:\repos\red-hat

APPENDIX C. USING AMQ BROKER WITH THE EXAMPLES

The Red Hat build of Apache Qpid ProtonJ2 examples require a running message broker with a queue named **hello-world-example**. Use the procedures below to install and start the broker and define the queue.

C.1. INSTALLING THE BROKER

Follow the instructions in *Getting Started with AMQ Broker* to [install the broker](#) and [create a broker instance](#). Enable anonymous access.

The following procedures refer to the location of the broker instance as **<broker-instance-dir>**.

C.2. STARTING THE BROKER

Procedure

1. Use the **artemis run** command to start the broker.

```
$ <broker-instance-dir>/bin/artemis run
```

2. Check the console output for any critical errors logged during startup. The broker logs **Server is now live** when it is ready.

```
$ example-broker/bin/artemis run
```

[illegible]

Red Hat AMQ <version>

```
2020-06-03 12:12:11,807 INFO [org.apache.activemq.artemis.integration.bootstrap]
AMQ101000: Starting ActiveMQ Artemis Server
```

■ ■ ■

```
2020-06-03 12:12:12,336 INFO [org.apache.activemq.artemis.core.server] AMQ221007:
Server is now live
```

...

C.3. CREATING A QUEUE

In a new terminal, use the **artemis queue** command to create a queue named **hello-world-example**.

```
$ <broker-instance-dir>/bin/artemis queue create --name hello-world-example --address hello-world-example --auto-create-address --anycast
```

You are prompted to answer a series of yes or no questions. Answer **N** for no to all of them.

Once the queue is created, the broker is ready for use with the example programs.

C.4. STOPPING THE BROKER

When you are done running the examples, use the **artemis stop** command to stop the broker.

```
$ <broker-instance-dir>/bin/artemis stop
```

Revised on 2026-02-19 21:45:55 UTC